

```
# The header we defined before will come here

paths:
  /movies:
    get:
      description: Returns all movies from the system
      operationId: findMovies
      parameters:
        - name: tags
          in: query
          description: Tags used to filter the result
          required: false
          type: array
          items:
            type: string
          collectionFormat: csv
        - name: limit
          in: query
          description: maximum number of results to return
          required: false
          type: integer
          format: int32
      responses:
        '200':
          description: A list of movies
          schema:
            type: array
            items:
              $ref: '#/definitions/Movie'

definitions:
  Movie:
    type: object

properties:
  movie_id:
    type: string
    description: Unique identifier representing a specific
Movie.
  title:
    type: string
    description: Title of the movie.
  description:
    type: string
    description: Description of the movie.
  rating:
    type: number
    description: Rating of the movie.
  price:
    type: number
    description: Price of the Movie at the store.
  image:
    type: string
    description: Image URL containing the movie poster.
```

```
/movies/{movie_id}:
  get:
    description: Returns a single movie by its movie_id
    parameters:
      - name: movie_id
        in: path
        required: true
        description: The movie's movie_id
        type: string
    responses:
      200:
        description: A Movie
        schema:
          $ref: '#/definitions/Movie'
      404:
        description: The Person does not exists.

/movies:
  post:
    description: Add a new movie to the collection
    operationId: addMovie
    parameters:
      - name: movie
        in: body
        description: The Movie JSON you want to post
        schema:
          $ref: '#/definitions/Movie'
    responses:
      '201':
        description: Movie created successfully
        schema:
          $ref: '#/definitions/Movie'
```

Swagger is portable and handy to use. You can import it to the AWS API gateway as well.

Modern application development leverages containerisation and a software architecture style that reduces the risks of services crashing due to unexpected exits. It allows developers to put their code into lightweight, platform-agnostic packages that can be easily moved across the infrastructure. Developers get more benefits when they deploy a specific piece of an application independently, without having to bother about the entire system. **END** 🐧

References

- [1] <http://microservices.io/>
- [2] <https://konghq.com/>
- [3] <https://aws.amazon.com/api-gateway/>
- [4] <https://swagger.io/>

 By: Radhakrishnan Ramakrishnan

The author is a DevOps engineer with Dattatreya Consultancy Services Pvt Ltd, Madurai. He has expertise in Linux, OpenStack, AWS, Web development and Hadoop administration.